

Encoder report — Dashcam Video Integrity

The encoder is a web page that records dashcam video from the laptop webcam (or a video file), cuts it into 10-second segments, computes a SHA-256 hash for every segment and perceptual hashes (pHash + dHash) twice per second, and sends the hashes live to Supabase. It keeps working without a network: everything is queued in the browser and uploaded later without losses or duplicates.

1. Install and run

Encoder and decoder run on **one server and one port**: `http://localhost:8000` shows the encoder, and the **Encoder | Decoder** buttons at the top switch between the two pages (`/` and `/decoder/`).

Requirements: [Node.js LTS](#) (only to build the encoder page), Python ≥ 3.11 , Google Chrome, a Supabase project, and `ffmpeg` only for the evaluation scripts.

Supabase (once): open the project → SQL Editor → paste all of `supabase/schema.sql` → Run. Then Settings → API: copy the Project URL and the publishable (anon) key.

macOS / Linux

```
# once (rebuild only if encoder/ code changes)
cp encoder/.env.example encoder/.env          # VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY
cp decoder/.env.example decoder/.env          # SUPABASE_URL, SUPABASE_ANON_KEY
cd encoder && npm install && npm run build && cd ..
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# every time: one command, one port
uvicorn decoder.main:app                       # open http://localhost:8000 in Chrome

# tests
cd encoder && npm test && SUPABASE_IT=1 npm test && cd ..
python -m pytest
```

Windows (PowerShell)

```
copy encoder\.env.example encoder\.env        # edit both with Notepad
copy decoder\.env.example decoder\.env
cd encoder; npm install; npm run build; cd ..
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

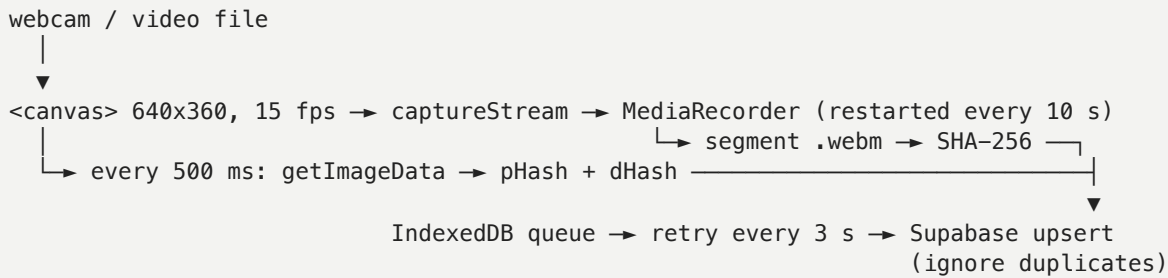
uvicorn decoder.main:app                       # open http://localhost:8000 in Chrome

cd encoder; npm test; $env:SUPABASE_IT=1; npm test; cd ..
python -m pytest
```

`localhost` is a secure context, so the camera works without HTTPS.

Using it (Encoder page, `/`): type a Driver ID → choose Webcam or Video file → Start. The Status table shows frames, hashes, rows sent, pending rows, network state and upload latency. "Simulate offline" stops the uploads. Recorded segments are listed at the bottom with a Download link (used to test the decoder). Keep the tab visible while recording (see limitations).

2. How it works (and why)



Requirement	Implementation	Why
Frame acquisition	<code>getUserMedia</code> (webcam) or a <code><video></code> playing a file; each frame is drawn on a canvas at 15 fps (<code>src/recorder.ts</code>)	One code path for both sources; the canvas is exactly what gets recorded and hashed
Video composed from the frames	<code>canvas.captureStream(15)</code> → <code>MediaRecorder</code> (WebM/VP8 in Chrome). A new recorder every 10 s	Each segment is a standalone playable file, so it has its own SHA-256 and can be verified alone
Hash generation	SHA-256 of every segment (<code>crypto.subtle</code>). Every 500 ms pHash and dHash of the canvas (<code>src/hash.ts</code>)	SHA-256 = exact integrity (any change breaks it). Perceptual hashes survive re-encoding, resizing, brightness, noise
Same hashes as the decoder	<code>hash.ts</code> and <code>decoder/fingerprint.py</code> implement the same algorithm: grayscale → 32x32 area resize → pHash (DCT 8x8 > median) / dHash (9x8 gradients)	A Vitest test hashes 3 PNG images in JS and compares with the Python hashes (≤ 4 bits; it passes)
Live transmission	Rows go to Supabase in batches every 3 s (<code>src/queue.ts</code> , <code>src/supabase.ts</code>)	Simple; 3 s delay is acceptable for a dashcam record
Network-loss handling	Every row is first saved in IndexedDB ; it is deleted only after the server confirmed it. Retry every 3 s and on the browser <code>online</code> event	Nothing is lost if the network or the server fails, or if the page is reloaded
No duplicates	Primary keys (<code>trip_id</code> , <code>sample_idx</code>) and (<code>trip_id</code> , <code>seq</code>) + <code>upsert(..., { ignoreDuplicates: true })</code> (= <code>INSERT ... ON CONFLICT DO NOTHING</code>). Locally the queue key is the same primary key	Re-sending is harmless and the order does not matter, so the retry logic stays trivial
Deletion of expired data	Every minute: local segments older than N minutes (default 10) are deleted from IndexedDB, and the server function <code>purge_expired(hours)</code> deletes rows older than N hours (default 24)	Limited local storage; old records do not stay forever
Security	Supabase RLS: the public key can only <code>insert</code> and <code>select</code> . No update/delete policy → the record is append-only. Deletion only through <code>purge_expired</code> (security definer, minimum 1 h)	Nobody with the public key can change or erase recent evidence

Positions in time: each fingerprint stores `t_ms` (elapsed time for the webcam, `video.currentTime` for a file). The decoder aligns by `t_ms`, not by index, because browser timers jitter.

3. Results

Automated tests (`npm test`, `SUPABASE_IT=1 npm test`): 10/10 pass.

- Hash parity JS vs Python on 3 test images (≤ 4 bits; exact on the same 32x32 input).
- Queue keeps rows while offline and when the server is unreachable; empties after reconnect.
- Same batch sent twice → no error, no duplicates (fake server and **real Supabase**).

- Real Supabase: the public key cannot update or delete; `purge_expired` is callable.

Network tests (real Supabase, values from the database — `eval/results/network_tests.md`):

Test	Rows queued	Result
Wi-Fi off during a whole file-mode trip (e5488849 , 83.6 s)	178 (1 trip + 9 segments + 168 fingerprints)	Uploaded in one batch after reconnect; 0 gaps in <code>sample_idx</code> and <code>seq</code>
Wi-Fi off during a webcam trip (d284be5b , 19.1 s)	39 (1 + 2 + 36)	Uploaded in one batch after reconnect; 0 gaps
Same trip online (a2280b49) for reference	—	Rows arrive spread over the 83 s of recording (live)
Duplicate sends (integration test)	—	Second send ignored, first version kept, 0 duplicates

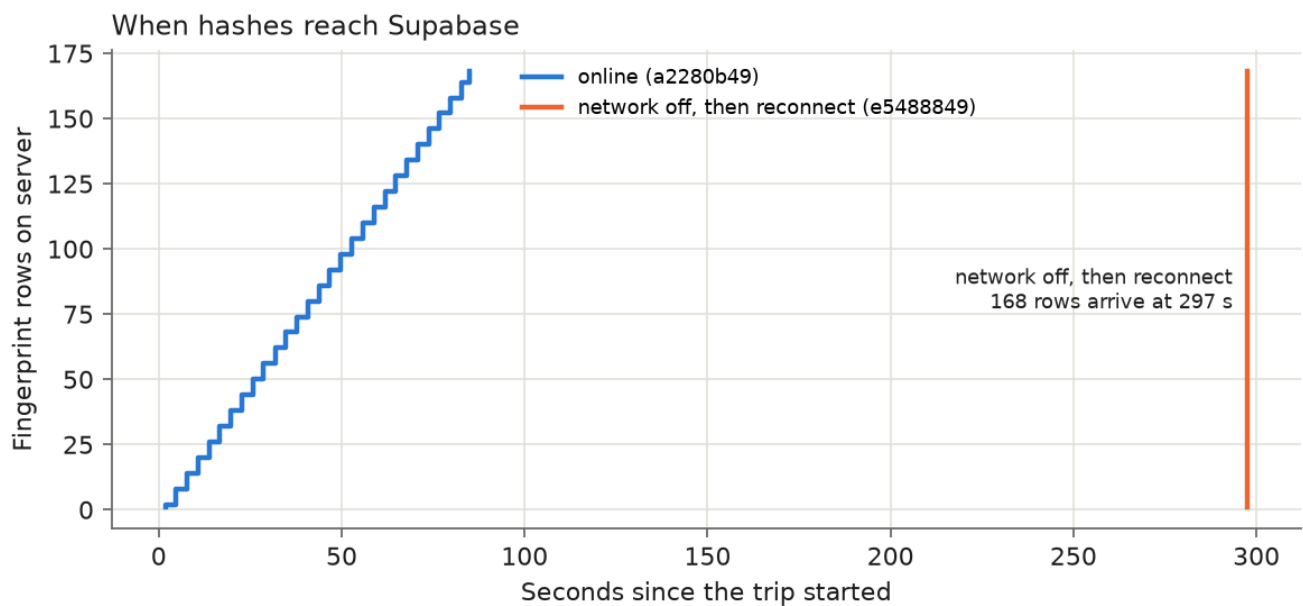


Figure 1 — When fingerprint rows reach Supabase (server arrival time, from the database). Online, rows arrive in small batches every 3 s while recording. With the network off, all 168 rows of the trip stay in the browser queue and arrive together when the network comes back (at 297 s here).

Sampling regularity: with the tab visible, samples are 469–531 ms apart (trip a2280b49). In an early trip where the tab was hidden for a while, spacing was 197–1007 ms — no samples were lost (0 gaps) but timing was irregular, which is why the decoder matches by time.

4. Limitations

- **Chrome recommended.** Chrome records WebM; Safari records MP4 (not tested).
- **The tab must stay visible** while recording: browsers throttle timers in background tabs (fewer frames, irregular samples).
- **No user authentication** (driver ID typed by hand): anyone with the public key can insert rows and read all trips, and can purge data older than 1 hour. Fine for a prototype, not for production.
- Local storage is the browser's: if the user clears site data before reconnecting, pending rows are lost.
- The video itself is only stored locally (10 min by default); only hashes go to the server.
- The encoder page is served as a static build (`encoder/dist`): after changing `encoder/` code (or `encoder/.env`) run `npm run build` again. `npm run dev` (port 5173) still works for development.
- Hashing every 500 ms runs on the main thread (~640×360 pixels); fine on a laptop, may be slow on old phones.